Level Up Coding · Follow publication

Member-only story

Level Up Your Claude Code with This CLAUDE.md

Make Claude Code Like a Senior Engineer with CLAUDE.md

8 min read · 4 days ago

Youssef Hosni Follow

Listen Share More

If you've been using Claude code, you've probably experienced both sides of it. On one hand, it can generate surprisingly good code, reason through tricky problems, and even help debug complex issues.

On the other hand, it can sometimes move too fast, skip planning, declare something "done" a bit too early, or apply a quick fix instead of addressing the root cause.

Most people interact with Claude in a purely prompt-driven way. They ask for a feature, review the output, make corrections, and repeat. But what's missing is a consistent engineering workflow. There's no built-in habit of planning first, verifying thoroughly, documenting lessons, or maintaining long-term discipline across sessions.

> *That's where a **claude.md** file changes everything.*

Instead of telling Claude what to do in each individual prompt, you define how it should operate as an engineer. You encode planning rules, debugging standards,

verification requirements, and quality principles into a single reusable document. From that point on, every task is executed within that structure.

In this post, I'll show you what **Claude.md** (or **agent.md**) actually is, break down a practical example line by line, and explain how you can use it to significantly improve the quality and reliability of Claude's coding output.

If you're using Claude for serious development work, this small file can make a surprisingly big difference.

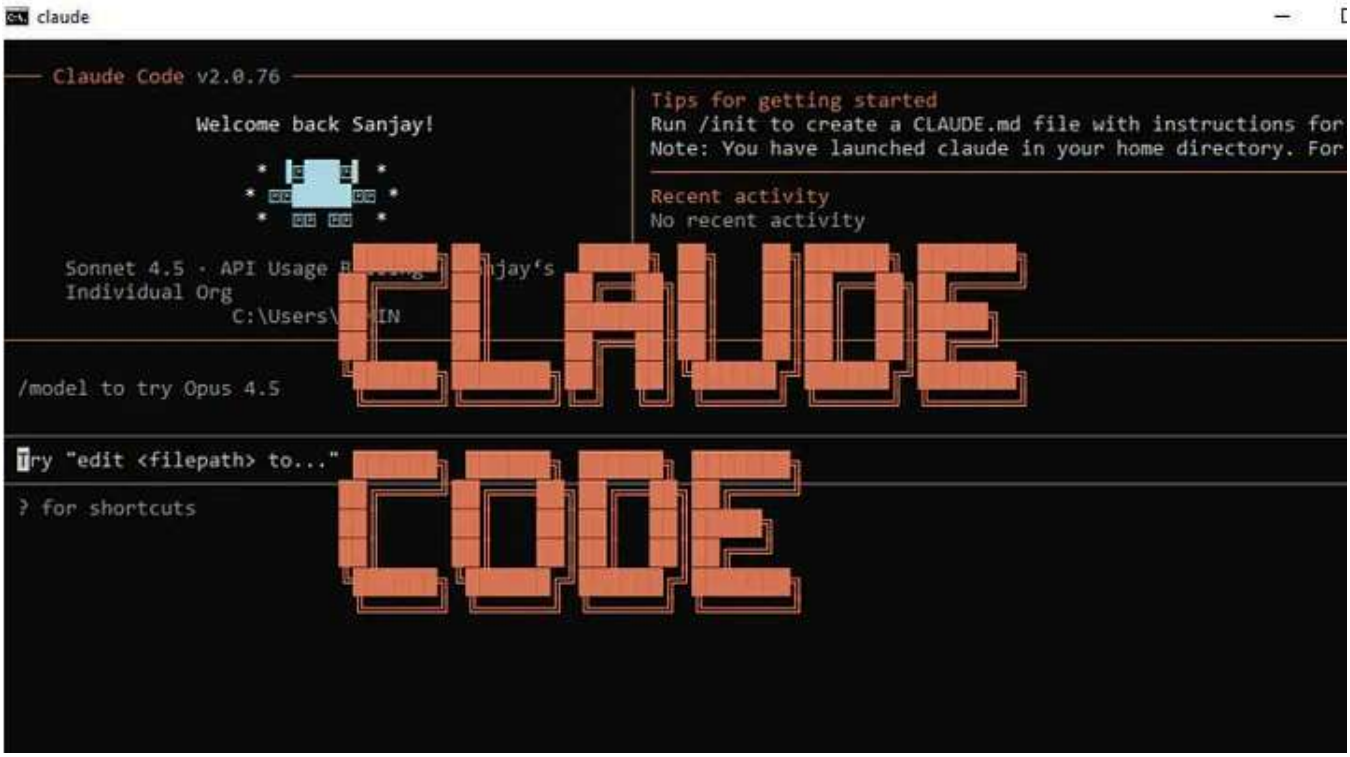


Table of Contents:

1. What is Claude.md (or agent.md), and Why Is It Important?

2. Breaking Down the CLAUDE.md File

3. How to Use It (and the Full Prompt)?

Everything I've Written, One Button Away, With 40% Off

Announcing My Ebooks Bundle + 50% Discount to my Followers

yousefhosni.medium.com

1. What is Claude.md (or agent.md), and Why Is It Important?

When most people start using Claude for coding, they focus entirely on prompts. They try to phrase requests better, add more context, or refine instructions when the output isn't what they expected. But over time, something becomes clear: the problem is rarely intelligence. Its structure.

Claude is capable of writing solid code, reasoning through complex systems, and even debugging multi-step issues. What it doesn't automatically do is enforce a disciplined engineering workflow. It doesn't naturally stop to plan before implementing. It doesn't always verify thoroughly before declaring something "done." And it certainly doesn't maintain long-term project habits unless you explicitly tell it to.

That's where a **Claude.md** (or **agent.md**) file comes in!

At its core, **Claude.md** is not a task description. It's not a feature spec. It's not documentation for your project. Instead, it is a behavioral contract for your AI assistant. It defines how the model should approach work, not what it should build. You are essentially encoding your engineering standards into a reusable framework that Claude follows across sessions.

Think of it as giving Claude an internal operating system.

Rather than repeating instructions like "**plan first**," "**don't rush**," "**fix the root cause**," or "**run tests before marking complete**," you formalize those expectations once. From that point forward, every task inherits that structure. This transforms Claude from a reactive tool that responds to prompts into a more structured collaborator that operates within clear engineering principles.

This becomes especially important as projects grow in complexity. Small tasks can tolerate loose execution. But once you're dealing with architectural decisions, multiple moving parts, bug triaging, or production-level reliability, unstructured

generation becomes expensive. Mistakes compound. Quick fixes introduce hidden issues. “Looks correct” replaces “verified correct.”

If you’re using Claude casually, you might never feel the need for this. But if you’re building real systems, maintaining codebases, or trying to reduce friction in long-running projects, having a clear behavioral framework is not optional. It’s the difference between reactive coding and intentional engineering.

2. Breaking Down the CLAUDE.md File

Let’s walk through the actual **Claude.md** step by step. I’ll quote each section and then explain why it matters and what it changes in practice.

2.1. Workflow Orchestration

```
### 1. Plan Mode Default
- Enter plan mode for ANY non-trivial task (3+ steps or architectural decisions
- If something goes sideways, STOP and re-plan immediately – don't keep pushing
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity
```

> *This is the backbone of the entire file.*

By default, most LLM interactions skip planning. You ask for a feature, and the model immediately starts coding. That works for small, contained tasks. But the moment you introduce architectural choices, dependencies, or multi-step workflows, skipping planning becomes expensive.

This section forces a behavioral shift. It says: if the task is even moderately complex, pause and think first. Write the plan. Clarify assumptions. Define structure.

The most important line here might be: *“If something goes sideways, STOP and re-plan immediately.”* LLMs tend to patch forward. They rarely step back unless explicitly instructed to do so. This rule prevents cascading mistakes by introducing structured reassessment.

```
### 2. Subagent Strategy

- Use subagents liberally to keep main context window clean
- Offload research, exploration, and parallel analysis to subagents
- For complex problems, throw more compute at it via subagents
- One task per subagent for focused execution
```

> *This section addresses a subtle but critical limitation: context overload.*

When everything happens in one continuous thread — research, debugging, planning, implementation — the quality of reasoning degrades. By encouraging subagents, the file promotes modular thinking.

Each subagent handles one clearly defined task. That mirrors how engineers break down work: isolate concerns, solve them independently, then integrate results.

It's not just about performance. It's about clarity. A clean context leads to cleaner reasoning.

```
### 3. Self-Improvement Loop

- After ANY correction from the user: update `tasks/lessons.md` with the patter
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate on these lessons until mistake rate drops
- Review lessons at session start for relevant project
```

> *This is where the workflow becomes adaptive.*

Most AI workflows are stateless. A **mistake** happens, you correct it, and then the session moves on. There's no long-term memory of patterns.

This section changes that. It forces the assistant to document errors and convert them into rules. Over time, that builds project-specific intelligence.

It's essentially teaching the assistant to reflect. That alone can dramatically reduce the number of repeated mistakes in long-running projects.

```
### 4. Verification Before Done

- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: “Would a staff engineer approve this?”
- Run tests, check logs, demonstrate correctness
```

> *This section raises the quality bar.*

LLMs are excellent at producing code that looks correct. But “looks correct” is not the same as verified correctness. This block enforces evidence.

Requiring diffs, tests, logs, and self-evaluation simulates a code review mindset. The line about asking whether a staff engineer would approve is particularly powerful — it reframes the task from “generate something plausible” to “meet senior-level standards.”

It introduces professional accountability into the workflow.

```
### 5. Demand Elegance (Balanced)
- For non-trivial changes: pause and ask "is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now, implement the elegant s
- Skip this for simple, obvious fixes — don't over-engineer
- Challenge your own work before presenting it
```

> *This section is about judgment.*

Without this rule, the assistant might either produce quick patches or overly abstract solutions. The instruction here is balanced: pursue elegance when necessary, but don’t complicate simple problems.

It encourages thoughtful refinement without drifting into unnecessary complexity.

That balance is what separates mature engineering from reactive coding.

```
### 6. Autonomous Bug Fixing
- When given a bug report: just fix it. Don't ask for hand-holding
- Point at logs, errors, failing tests — then resolve them
- Zero context switching required from the user
- Go fix failing CI tests without being told how
```

> *This section shifts responsibility.*

Instead of asking for clarification at every step, the assistant is instructed to investigate independently. Look at logs. Identify failures. Trace root causes.

This mimics how experienced engineers approach bugs. They don't immediately ask what to do. They diagnose first.

It reduces friction and makes the assistant feel proactive rather than reactive.

Task Management & Core Principles

```
1. Plan First
2. Verify Plan
3. Track Progress
4. Explain Changes
5. Document Results
6. Capture Lessons
```

This introduces operational discipline. Planning is documented. Progress is tracked. Changes are explained. Lessons are preserved.

It ensures traceability rather than chaotic iteration.

And finally:

```
- Simplicity First
- No Laziness
- Minimal Impact
```

> *These are cultural guardrails.*

"Simplicity First" prevents unnecessary abstraction.
"No Laziness" forces root-cause analysis instead of temporary patches.
"Minimal Impact" protects stability by limiting the change surface area.

Together, these principles define not just *how* the assistant works, but *how it thinks*.

3. How to Use It (and the Full Prompt)?

Place **CLAUDE.md** at the root of your repository. When starting a session, tell Claude:

> Follow the rules defined in CLAUDE.md for this project.

Here is the full content:

```
## Workflow Orchestration

### 1. Plan Mode Default
- Enter plan mode for ANY non-trivial task (3+ steps or architectural decisions
- If something goes sideways, STOP and re-plan immediately – don't keep pushing
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity

### 2. Subagent Strategy
- Use subagents liberally to keep main context window clean
- Offload research, exploration, and parallel analysis to subagents
- For complex problems, throw more compute at it via subagents
- One task per subagent for focused execution

### 3. Self-Improvement Loop
- After ANY correction from the user: update `tasks/lessons.md` with the patter
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate on these lessons until mistake rate drops
- Review lessons at session start for relevant project

### 4. Verification Before Done
- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: "Would a staff engineer approve this?"
- Run tests, check logs, demonstrate correctness

### 5. Demand Elegance (Balanced)
- For non-trivial changes: pause and ask "is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now, implement the elegant s
```

```
- Skip this for simple, obvious fixes – don't over-engineer
- Challenge your own work before presenting it

### 6. Autonomous Bug Fixing
- When given a bug report: just fix it. Don't ask for hand-holding
- Point at logs, errors, failing tests – then resolve them
- Zero context switching required from the user
- Go fix failing CI tests without being told how

## Task Management

1. **Plan First**: Write plan to `tasks/todo.md` with checkable items
2. **Verify Plan**: Check in before starting implementation
3. **Track Progress**: Mark items complete as you go
4. **Explain Changes**: High-level summary at each step
5. **Document Results**: Add review section to `tasks/todo.md`
6. **Capture Lessons**: Update `tasks/lessons.md` after corrections

## Core Principles

- **Simplicity First**: Make every change as simple as possible. Impact minimal
- **No Laziness**: Find root causes. No temporary fixes. Senior developer stand
- **Minimal Impact**: Changes should only touch what's necessary. Avoid introdu
```

When you start using Claude (or any strong coding LLM) for real projects, you quickly notice something: the model is powerful, but the *workflow* isn't always structured.

Sometimes it jumps straight into implementation.
Sometimes it skips planning.
Sometimes it "marks done" without truly verifying.

That's where a **CLAUDE.md** (or **agent.md**) file comes in.

In this post, we have covered:

1. What `Claude.md` (or `agent.md`) actually is — and why it matters
2. A breakdown of the provided `claude.md`
3. How to use it in practice — including the full prompt

This blog is a personal passion project, and your support helps keep it alive. If you would like to contribute, there are a few great ways:

- Subscribe. A paid subscription to my newsletter helps sustain my writing and gives you access to additional content.
- Grab a copy of my book Bundle. Get my 7 hands-on books and roadmaps for only 40% of the price

Thanks for reading, and for helping support independent writing and research!

Are you looking to start a career in data science and AI, but do not know how? I offer data science mentoring sessions and long-term career mentoring:

- 1–1 Mentoring sessions
- Long-term mentoring

Claude Code AI Agents

Follow

Open in app ↗

Medium

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Follow

Written by Youssef Hosni

32K followers · 95 following

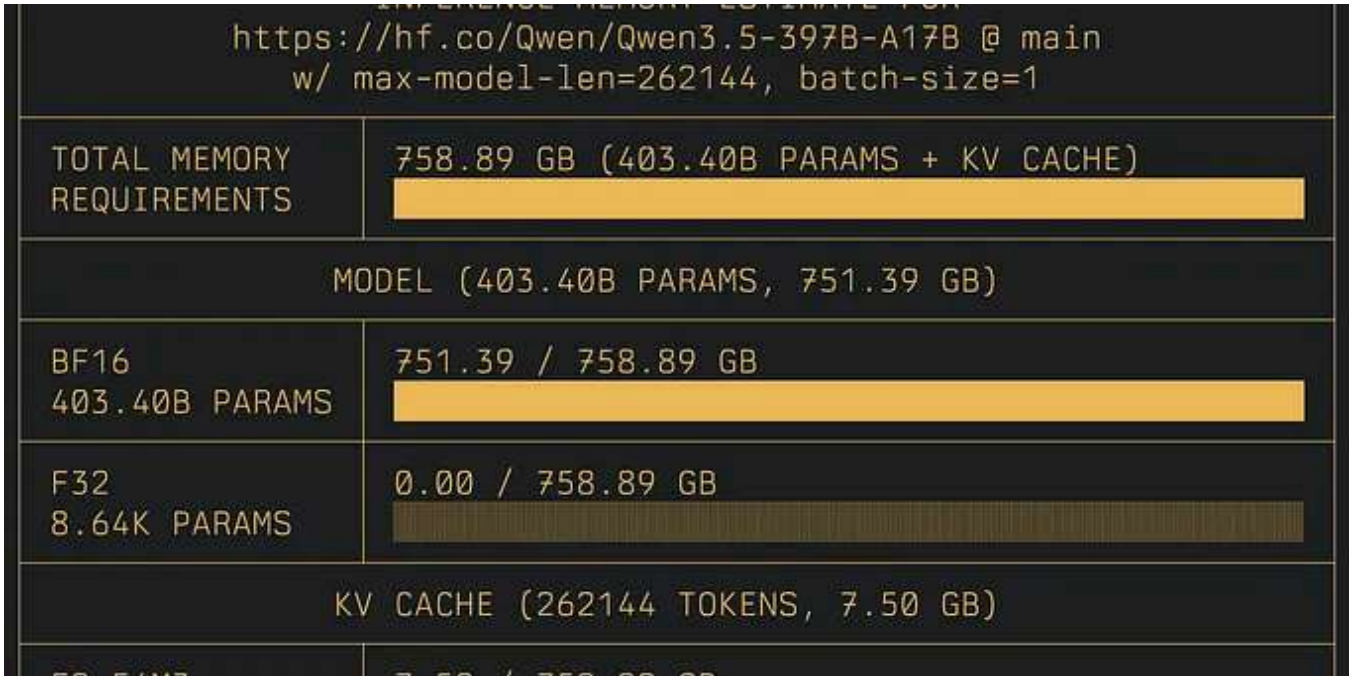
Applied AI @ GreenStep | AI Researcher @ Aalto University | Founder @ To Data & Beyond | Subscribe to my Newsletter: https://youssefh.substack.com/

No responses yet

Holger Kreissl

What are your thoughts?

More from Youssef Hosni and Level Up Coding

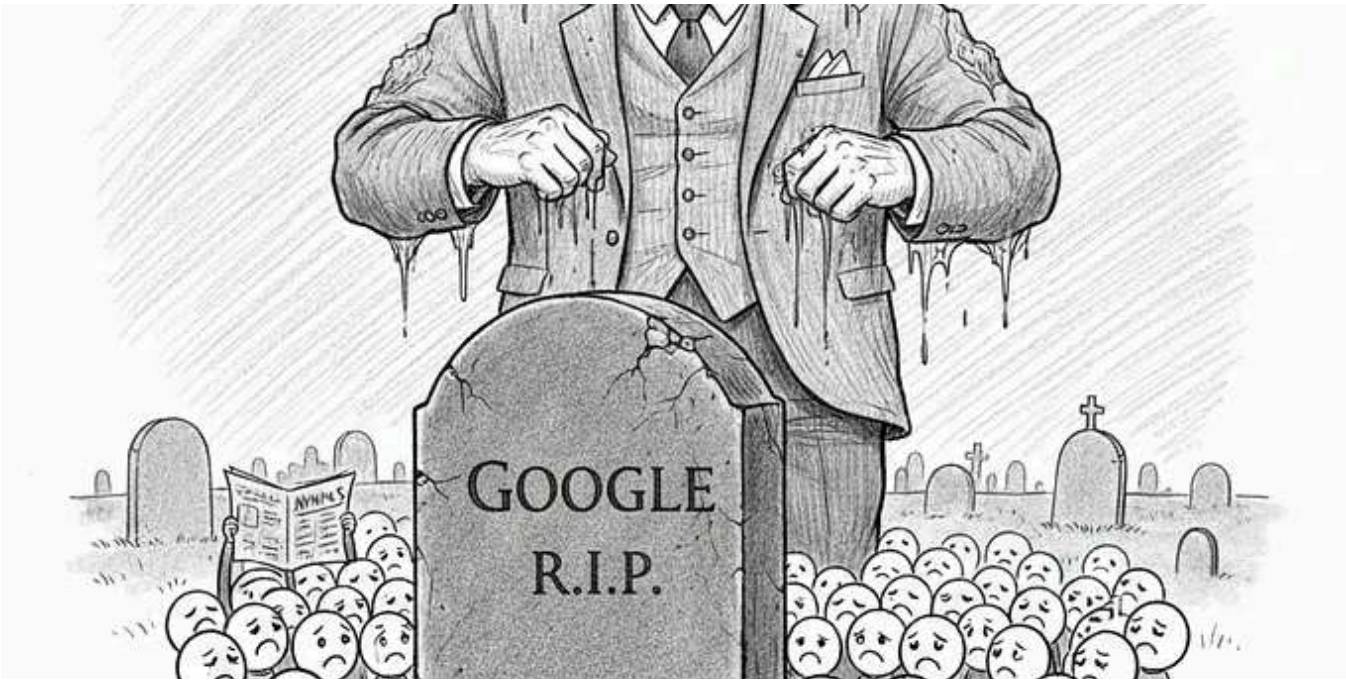


In To Data & Beyond by Youssef Hosni

Stop Calculating LLM Memory Requirement: Use hf-mem!

If you've worked with large models from Hugging Face, you've probably asked yourself this question:

Feb 19 170

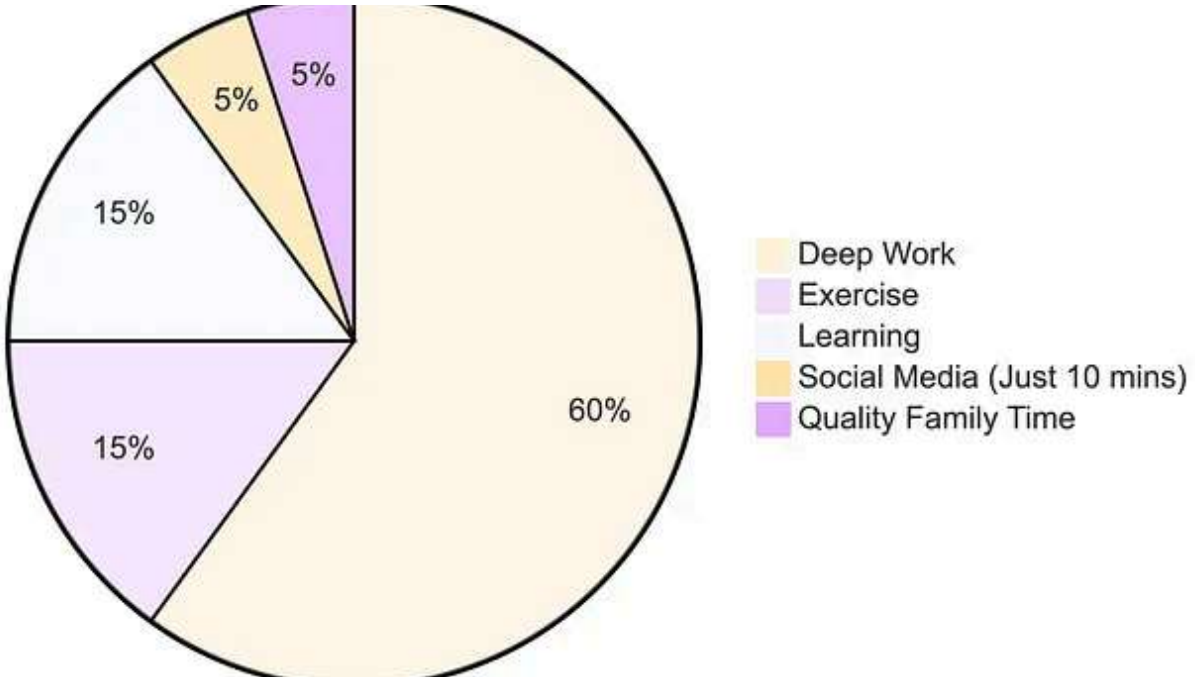


In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine inside someone else's car.

Feb 17 4K 136



In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

Dec 28, 2025 8.5K 326

In Level Up Coding by Youssef Hosni

Use GLM-5 in Claude Code and Save 60% on Tokens

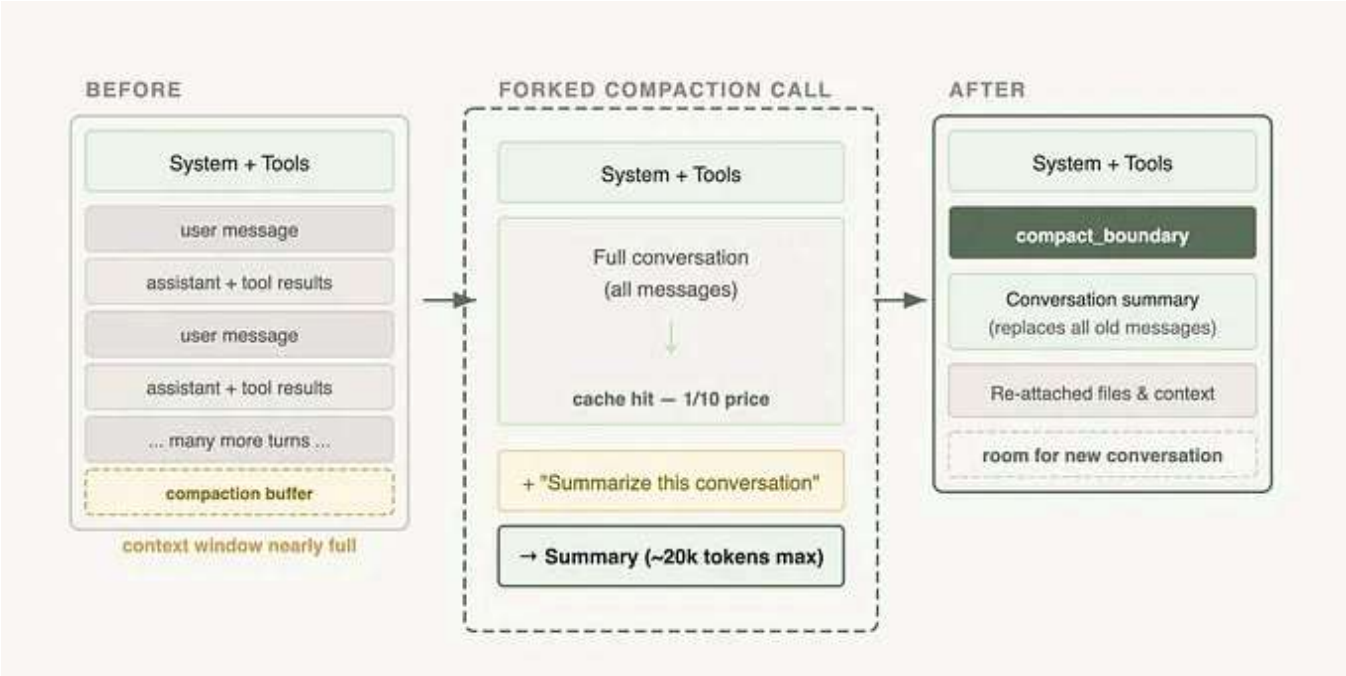
Running GLM-5 Inside Claude Code: Setup and Best Workflow

6d ago 231 1

See all from Youssef Hosni

See all from Level Up Coding

Recommended from Medium

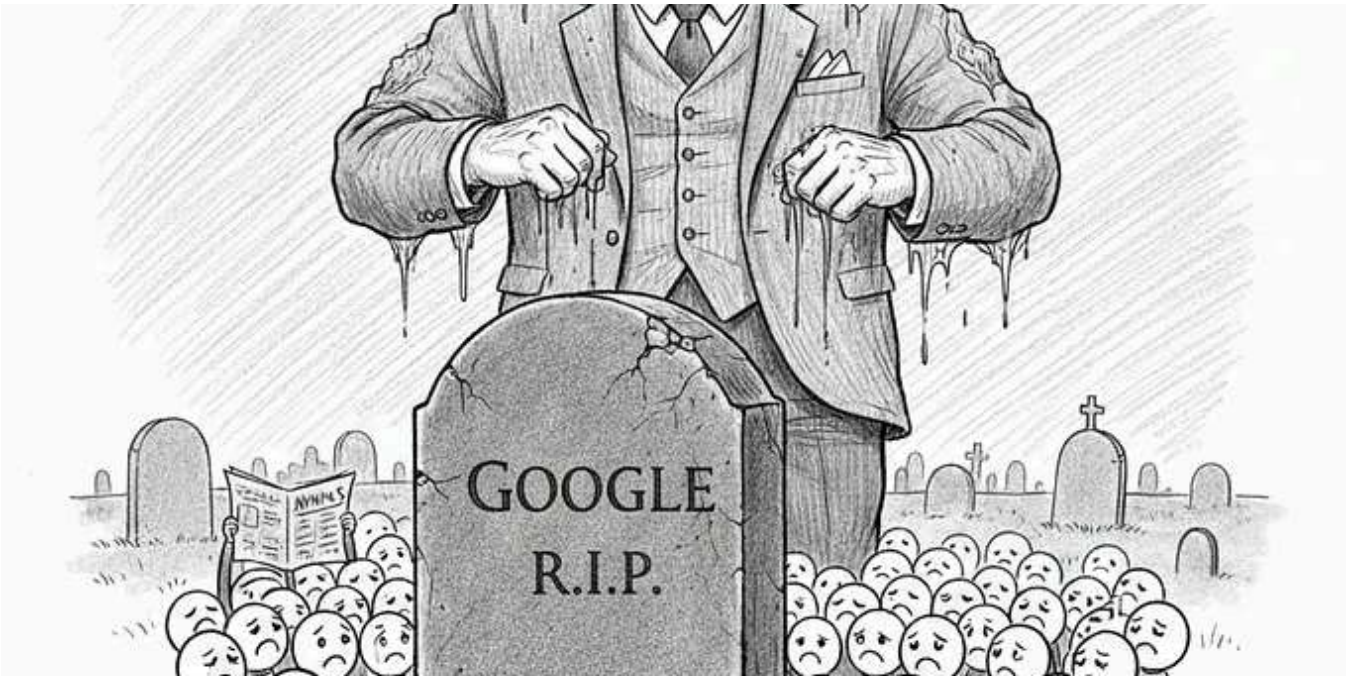


In AI Software Engineer by Joe Njenga

Anthropic Just Fixed the Biggest Hidden Cost in AI Agents (Automatic Prompt Caching)

With just one change, you can cut Claude API costs to 10 cents on the dollar and stop bleeding cash on every single API call.

Feb 20 729 16



In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine inside someone else's car.

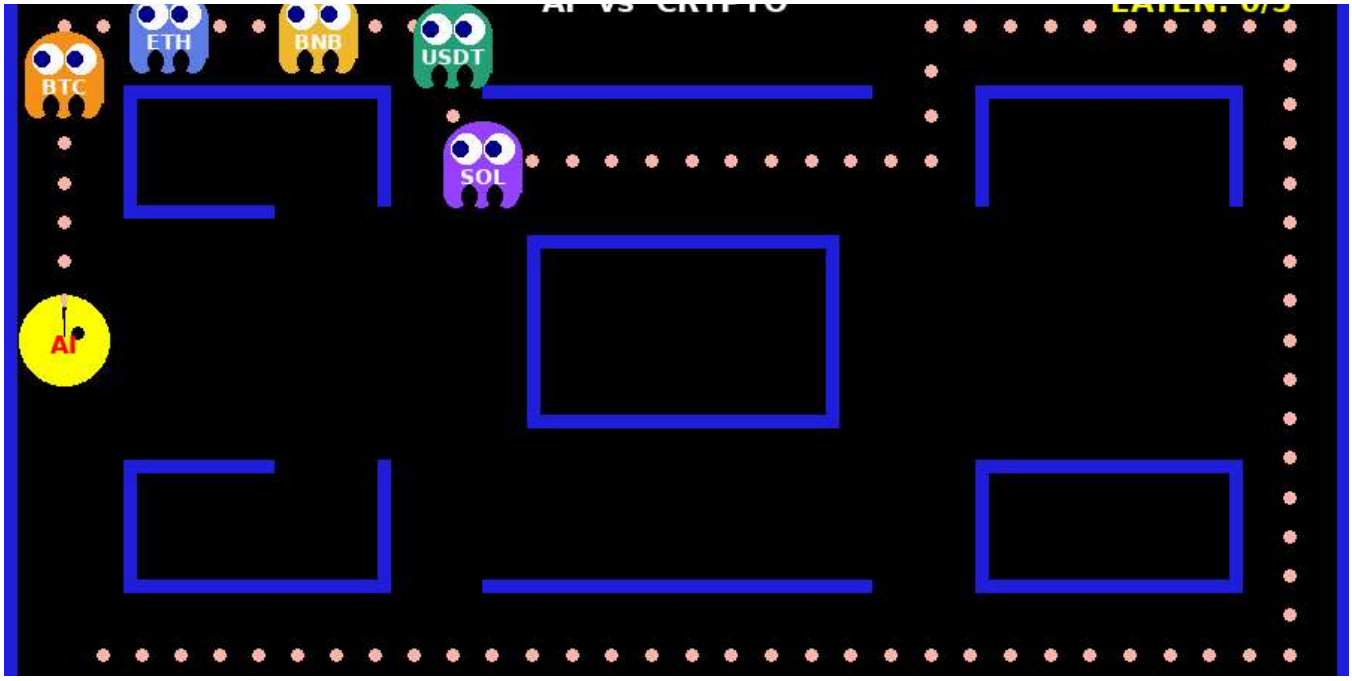
Feb 17 4K 136

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big Tech AI Subscriptions Forever |...

Something strange happened in early 2026. Apple stores started running low on Mac Minis. Tech forums exploded with setup guides. Developers...

Feb 15 3.5K 60

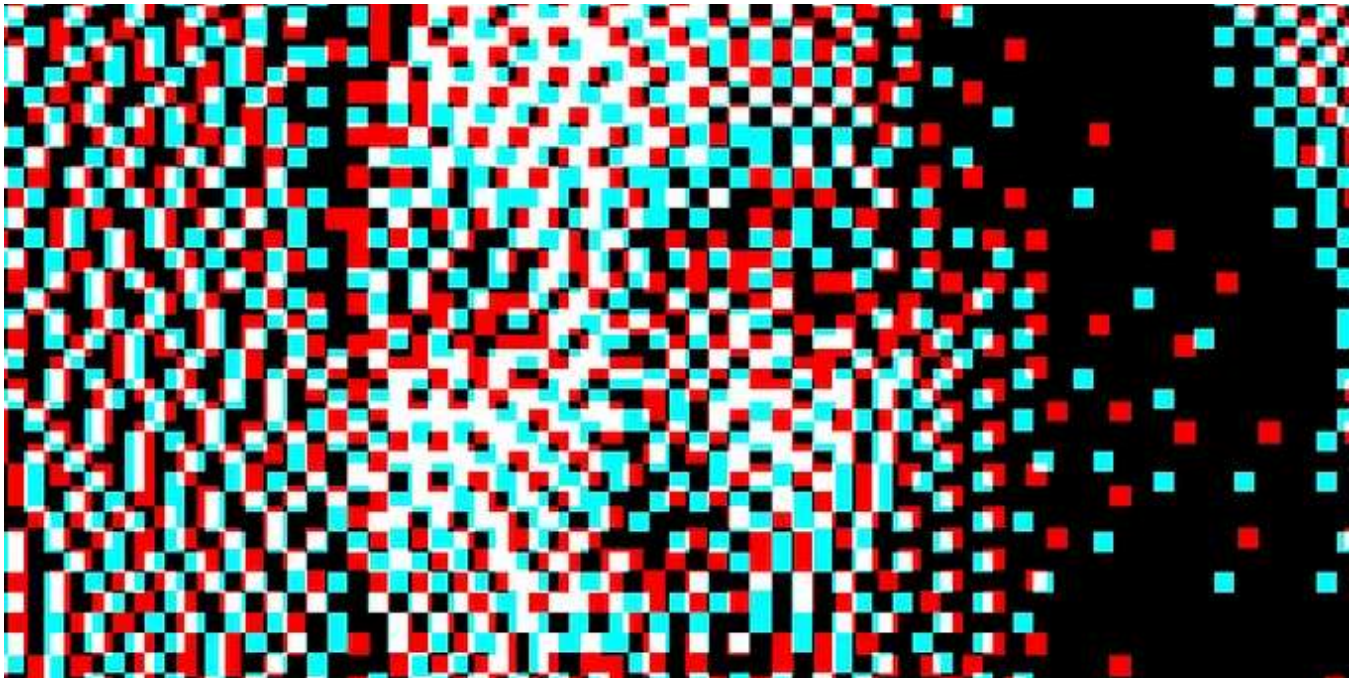


In AI Advances by Jose Crespo, PhD

Anthropic is Killing Bitcoin

The AI-native currency already exists — hiding in plain sight, outperforming crypto by six orders of magnitude.

Feb 17 2.7K 123

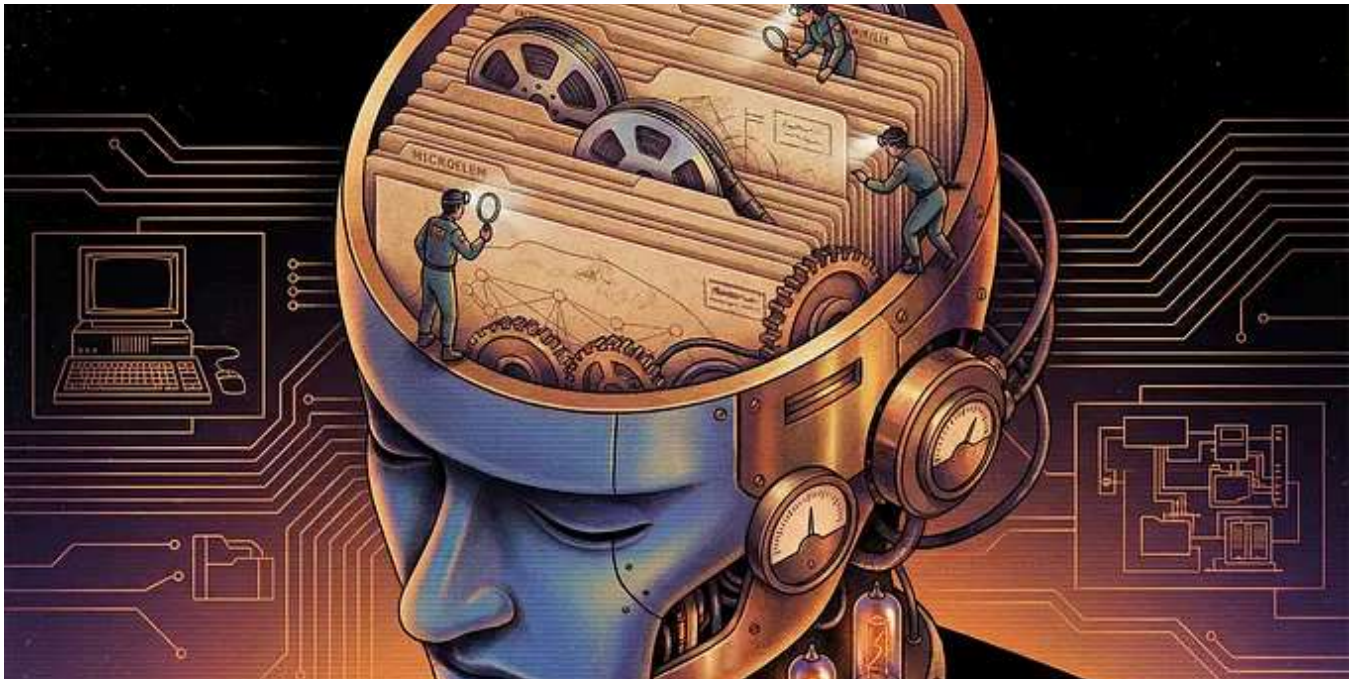


Marco Kotrotsos

The Agentic Engineering Playbook.

According to OpenClaw's Creator

Feb 17 210 8



Agent Native

OpenClaw Memory Systems That Don’t Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it… it’s not random.

Feb 20 132 2

See more recommendations